

Cloud_IOT Pedestal SW

ERP Integration Guide

Version 3.2 (covers SW v3.32) — June 20, 2026

Lika Digital d.o.o.

This document describes the complete API surface, webhook events, authentication model, and integration patterns for connecting an external ERP system to the Cloud_IOT Smart Pedestal management platform. It documents the complete live endpoint and event catalog, and is written as a concrete integration guide for the MarinaMaster ERP and its MyMarina end-user mobile application.

Contents

- MarinaMaster ERP — Integration Quick-Start Summary
- MVP Pilot — First Integration Set (Temperature, Berths, Camera)
- NFC Activation API (myMarina ERP)
- 1. Architecture Overview
- 2. Authentication
- 3. External API Gateway Setup
- 4. Endpoint Catalog (53 endpoints)
- 5. Webhook Event Catalog (38 events)
- 6. Core Integration Flows
- 7. Session Lifecycle & Billing
- 8. Contracts & Invoices
- 9. Webhook Payload Reference
- 10. Error Handling & Resilience
- 11. Security Considerations
- 12. Network & Cloudflare Configuration

- 13. Multi-Client Authentication (Service Accounts)
- 14. Reference Implementation (ERP-IOT)
- 15. Quick-Start Checklist
- 16. Data for the Marina Operator Persona
- 17. Data Exposed to the MyMarina End-User App

MarinaMaster ERP — Integration Quick-Start Summary

This section provides a high-level overview of all steps required for the MarinaMaster ERP management team to begin integration with Lika Digital's Cloud_IOT Pedestal SW. Each step references the detailed section later in this document.

Phase 1 — Network Connectivity

Step 1: Determine your network setup. Does MarinaMaster have a static public IP address? If yes, proceed with Option A. If no, proceed with Option B. (See Section 12 for full details.)

Step 2 (Option A — Static IP): Send your server's public IP address and the port your ERP application runs on to Lika Digital. No further action required from MarinaMaster — Lika Digital handles all Cloudflare and DNS configuration.

Step 3 (Option B — No Static IP): Install `cloudflared` on your server, create a tunnel, and send the tunnel ID to Lika Digital. Lika Digital will configure the DNS record. Full step-by-step instructions are in Section 12.3. This takes approximately 5 minutes.

Phase 2 — Credentials & Configuration

Step 4: Provide service account details to Lika Digital. Send the desired email address and password for your ERP's service account. Lika Digital will create the account on the Pedestal SW with role `api_client`. (See Section 13.)

Step 5: Provide your webhook URL. This is the HTTPS endpoint on the MarinaMaster side where the Pedestal SW will send real-time events (session started, session completed, sensor readings, alarms). (See Section 5 for event catalog.)

Step 6: Provide the list of required endpoints and events. Review the endpoint catalog (Section 4) and event catalog (Section 5). Tell Lika Digital which endpoints you need and whether each should be monitor (read-only) or `bidirectional` (read + write/control).

Step 7 (Optional): Provide IP whitelist or mTLS certificate. For additional security, provide your server's IP range or a client TLS certificate. Lika Digital will configure Cloudflare Access policies accordingly.

Phase 3 — Lika Digital Configuration

Once MarinaMaster provides the above information, Lika Digital will:

- Create the service account on the Pedestal SW.
- Configure the API Gateway with MarinaMaster's requested endpoints, events, and webhook URL.
- Configure Cloudflare DNS/tunnel routing (and Access policies if requested).
- Verify and activate the gateway.
- Provide MarinaMaster with the API base URL: `https://marina.lika.solutions`.

Phase 4 — MarinaMaster Implementation

Step 8: Implement authentication. Call POST `https://marina.lika.solutions/api/auth/service-token` with your service account credentials to obtain a JWT token. Cache it for 7 hours and refresh before expiry. (See Section 13.3.)

Step 9: Implement API polling. Call the gateway endpoints (e.g., GET `/api/ext/sessions/active`) with the JWT in the Authorization: Bearer header. Implement retry + stale-data fallback. (See Sections 6 and 10.)

Step 10: Implement webhook handler. Accept incoming POST events at your webhook URL. Validate the X-API-Key header. Persist events and return HTTP 200 within 5 seconds. (See Section 5.2 and 9.)

Step 11: Implement control actions (if needed). To approve, deny, or stop sessions from the ERP, call the bidirectional control endpoints. (See Section 6.2.)

Step 12: Implement billing sync. Listen for `session_completed` webhook events and query `/api/ext/billing/spending/detail` for reconciliation. (See Section 7.)

Phase 5 — Testing & Go-Live

Step 13: End-to-end testing. Lika Digital will run the pedestal simulator to generate test sessions. MarinaMaster verifies that webhook events arrive, API polling returns correct data, and control actions work.

Step 14: Go-live. Once both parties confirm the integration is working, the system is ready for production use.

Note: Summary of what MarinaMaster must provide to Lika Digital:

1. Network: Static IP + port, or cloudflared tunnel ID
2. Service account: desired email + password
3. Webhook URL (HTTPS)
4. Required endpoints (with monitor/bidirectional mode)
5. Required webhook events
6. (Optional) IP whitelist or mTLS certificate

MVP Pilot — First Integration Set

Before wiring the full endpoint and event catalog, MarinaMaster and Lika Digital will validate the integration with a small, concrete pilot. The pilot exercises every part of the pipeline — API key authentication, a per-endpoint toggle, a pull endpoint, a webhook push event, and a media stream — against three real use cases. Once these three work end-to-end, every other capability described in the API segment (Sections 4 and 5) can be enabled the same way, with no new integration mechanics to learn.

The three pilot use cases are:

- **1. Export temperature sensor state** — the cabinet temperature (Papouch TME sensor) is pushed to MarinaMaster in real time, including alarm severity.
- **2. Berth status per berth** — occupied / free status for each berth under a pedestal, available both on demand (pull) and as a real-time push.
- **3. Live camera per berth via the NUC** — MarinaMaster obtains a live view of a defined berth, either as the camera stream URL or as a single JPEG frame fetched through the NUC.

Note: All three are **monitor (read-only)** capabilities — no control or write access is required for the pilot. Authentication, base URL, and gateway activation are exactly as described in the Quick-Start Summary above (service token or API key in the `Authorization: Bearer` header; webhook secured with `X-API-Key`).

Use Case 1 — Export Temperature Sensor State

The cabinet temperature is read from a networked Papouch TME sensor by the NUC every 30 seconds and pushed to MarinaMaster as a `temperature_reading` webhook event. This is a **push-only** capability — there is no polling endpoint for temperature; MarinaMaster receives each reading as it is taken.

Event: `temperature_reading` (enable in the gateway's allowed events).

```
{
  "event": "temperature_reading",
  "data": {
    "pedestal_id": 1,
    "value": 47.2,
    "severity": "warning",
    "alarm": true,
    "temp_sensor_reachable": true,
    "last_temp_sensor_check": "2026-06-14T13:05:00Z",
    "timestamp": "2026-06-14T13:05:00Z"
  },
  "timestamp": "2026-06-14T13:05:01Z"
}
```

`severity` is `null` when the reading is in the normal band, `"warning"` or `"critical"` otherwise; `alarm` is `true` whenever `severity` is non-`null`. The alarm bands are documented in Section 9.4. If the sensor becomes unreachable the NUC emits a separate `temp_sensor_offline` alarm and `temp_sensor_reachable` reports `false`.

Use Case 2 — Berth Status per Berth (Occupied / Free)

Each berth under a pedestal carries an occupancy bit derived from the camera analysis on the NUC. MarinaMaster can read it on demand (pull) and also receive a push whenever it changes.

Pull: current occupancy for all berths under a pedestal.

```
GET /api/ext/pedestals/{pedestal_id}/berths/occupancy
Authorization: Bearer {api_key}

# pedestal_id may be the numeric DB id OR the Opta client id,
# e.g. GET /api/ext/pedestals/MAR_KRK ORM_01/berths/occupancy

200 OK
{
  "pedestal_id": "MAR_KRK ORM_01",
  "berths": [
    { "berth_id": 7, "berth_name": "A-12", "occupied": true,
      "last_analyzed": "2026-06-14T13:02:11Z" },
    { "berth_id": 8, "berth_name": "A-13", "occupied": false,
      "last_analyzed": "2026-06-14T13:02:11Z" }
  ]
}
```

occupied is true / false, or null with a note field when the berth has not been analyzed yet. A pedestal with no berth definitions returns "berths": [] plus a message. Requires the berths.occupancy_ext endpoint to be enabled (returns 503 if not).

Push: real-time occupancy change.

```
{
  "event": "berth_occupancy_updated",
  "data": {
    "pedestal_id": 1,
    "berths": [ { "berth_id": 7, "occupied": true } ]
  },
  "timestamp": "2026-06-14T13:02:12Z"
}
```

Note: The webhook delivers a deliberately slim occupancy shape (berth_id + occupied only). Internal analysis details — zone rectangles, camera URL, match scores, re-ID embeddings — are never sent to the ERP.

Use Case 3 — Live Camera per Berth via the NUC

Each pedestal has one camera that covers its berths, so a “defined berth” is viewed through its parent pedestal’s camera. MarinaMaster has two options, both served by the NUC at that marina:

Option A — Stream URL: the NUC returns the RTSP URL and current reachability; MarinaMaster connects to the stream directly.

```
GET /api/ext/pedestals/{pedestal_id}/camera/stream
Authorization: Bearer {api_key}

200 OK
{
  "pedestal_id": "MAR_KRK ORM_01",
  "stream_url": "rtsp://.../profile1",
  "reachable": true,
  "last_checked": "2026-06-14T13:04:00Z"
}
```

Option B — Single frame through the NUC: the NUC grabs one live JPEG from the camera and returns the image bytes, so MarinaMaster never needs RTSP access or direct network reach to the camera.

```
GET /api/ext/pedestals/{pedestal_id}/camera/frame
Authorization: Bearer {api_key}

200 OK    Content-Type: image/jpeg    (raw JPEG bytes)
```

Option A requires the `camera.stream_ext` endpoint; Option B requires `camera.frame_ext`. Either returns 503 with `{error, reason}` if the camera is not configured, not reachable, or the endpoint is disabled. For a pilot behind a Cloudflare tunnel, Option B is recommended — it travels over the same HTTPS channel as the rest of the API and needs no separate RTSP path.

Pilot Enablement Checklist

To run the pilot, Lika Digital enables the following on the gateway for MarinaMaster's API key (all read-only / monitor):

Use Case	Type	Catalog item to enable
1. Temperature	Event (push)	<code>temperature_reading</code>
2. Berth status	Endpoint (pull)	<code>berths.occupancy_ext</code>
2. Berth status	Event (push)	<code>berth_occupancy_updated</code>
3. Camera (URL)	Endpoint (pull)	<code>camera.stream_ext</code>
3. Camera (frame)	Endpoint (pull)	<code>camera.frame_ext</code>

Note: What success unlocks: the pilot proves the full mechanism — key auth, per-endpoint toggles, pull endpoints, webhook push, and media through the NUC. Once it is green, any of the remaining 53 endpoints and 38 events in Sections 4 and 5 (sessions, billing, power/water telemetry, breaker and load monitoring, diagnostics, contracts) can be exported by simply adding them to MarinaMaster's allowed lists — no new integration code on either side.

NFC Activation API (myMarina ERP)

This is a dedicated machine-to-machine API for the myMarina end-user app, **separate** from the gateway in sections 1–13. It lets myMarina pre-register an NFC tap; the socket then activates only when the boater physically plugs in their charger. The marina operator always retains highest-priority control over every session.

Prerequisite — Smart Mode must be ON

Smart Mode (Opta firmware v3.0.0) is the master switch that decides who controls a cabinet. It defaults **OFF** on every firmware boot, and while OFF the Opta runs **standalone** and ignores NUC commands — so sessions, NFC activation, auto-activate, load monitoring and billing do nothing. The whole NUC/ERP integration (including this NFC API and the gateway in sections 1–13) is only effective when Smart Mode is **ON** for that pedestal.

Field / endpoint	Meaning
smart_mode (bool)	Reported per pedestal in the health response and on every opta_status webhook/WS event. False = standalone (NUC control inert).
POST /api/pedestals/{cabinet_id}/smartmode	Operator/admin sets it ({"value": true false}). Publishes opta/cmd/smartmode to the cabinet.

Note: ERP guidance: before driving a pedestal, confirm `smart_mode == true` (from the health response or the latest `opta_status`). If it is false, the pedestal is operator-managed standalone hardware — surface that to the user rather than expecting activation to take effect. Smart Mode is enabled by the marina operator, not by the ERP.

Usage history & monthly reports (v3.31)

Each completed session is retained as usage history per socket/valve and is exposed read-only to the ERP. The marina operator opts each endpoint in from the API Gateway page (they appear under the **Usage History** category). Values come straight from the session record; when a session was not attributed to a customer (e.g. Smart Mode was off), the customer / NFC fields are simply empty.

Endpoint	Returns
GET /api/ext/pedestals/{id}/usage/history	Completed-session rows (socket/valve, start & end, kWh, litres, customer_id, customer_name, nfc_user_id). Optional query: resource, socket_id, month=YYYY-MM.
GET /api/ext/pedestals/{id}/usage/reports	List of available monthly report files (month, filename, size, generated_at).
GET /api/ext/pedestals/{id}/usage/reports/{YYYY-MM}	The plain-text monthly report for the pedestal (all sockets/valves, with totals). Generated on demand if it does not exist yet.

Note: Reports are protected: they are created automatically each month, kept on disk, and can be deleted **ONLY** by an operator-admin on the dashboard — never by the system and never via this API (the gateway permits GET only for these endpoints).

Authentication

Every `/api/nfc/` request authenticates with a static API key in the `X-API-Key` header (configured server-side as `ERP_API_KEY`). This is independent of the gateway JWT and its endpoint/event toggles. Missing or invalid key → 401; the feature being unconfigured on the server → 503.

```
POST https://marina.lika.solutions/api/nfc/scan
X-API-Key: {ERP_API_KEY}
Content-Type: application/json
```

Provisioning (operator side)

NFC tags are mapped to sockets by the marina operator in the dashboard (Control Center → Socket Settings → NFC) — one tag per socket; a tag already assigned elsewhere is rejected. The ERP does not provision tags; it only sends scans for tags that are already mapped.

Endpoints

Method & path	Purpose
POST <code>/api/nfc/scan</code>	Pre-register an NFC tap. Does NOT activate the socket.
GET <code>/api/nfc/session/{id}</code>	Live session status + spending data.
POST <code>/api/nfc/session/{id}/stop</code>	Remote stop (operator override still applies).

Activation flow

- Boater taps the socket's NFC tag in myMarina
ERP -> POST `/api/nfc/scan {nfc_tag_id, user_id}`
<- 200 `{status:"pending", expires_at = now + 5 min}`
- Boater plugs in WITHIN 5 minutes
Pedestal reports the plug-in over MQTT -> socket activates, the myMarina user is attached to the session.
(no tap, or tap expired -> socket stays idle; nothing happens)
- ERP polls GET `/api/nfc/session/{id}` for live kWh / cost
- Stop: operator (dashboard) OR ERP POST `/api/nfc/session/{id}/stop`

Note: The socket is NOT switched on by the scan. Activation happens exclusively when the Opta pedestal detects a physical plug-in. A scan with no plug-in inside 5 minutes simply expires.

Payloads

POST `/api/nfc/scan` — request & response:

```
// request
{ "nfc_tag_id": "04:A2:3B:1C", "user_id": "erp-12345" }

// 200
{
  "status": "pending",
  "message": "Please plug in your charger to activate the socket",
  "cabinet_id": "MAR_KRK ORM_01",
  "socket_id": "Q1",
  "berth_id": "VEZ_A1",
```

```
"expires_at": "2026-06-16T12:05:00Z"
}
```

GET /api/nfc/session/{id} — response (same shape the webhook sends):

```
{
  "session_id": 123,
  "cabinet_id": "MAR_KRK ORM_01",
  "socket_id": "Q1",
  "customer_id": "erp-12345",           // the myMarina user
  "status": "active",                 // "active" | "ended"
  "activated_at": "2026-06-16T12:01:30Z",
  "duration_minutes": 42.5,
  "energy_kwh": 2.45,
  "power_kw_current": 1.10,
  "estimated_cost": 0.74              // null if no tariff configured
}
```

Note: Scan status codes: 404 NFC tag not provisioned · 409 Socket already in use · 503 Socket unavailable (fault) · 401 missing/invalid key. Remote stop returns 409 “Session already ended by operator” if the operator already stopped it — a stopped session is never restarted.

Webhook (optional)

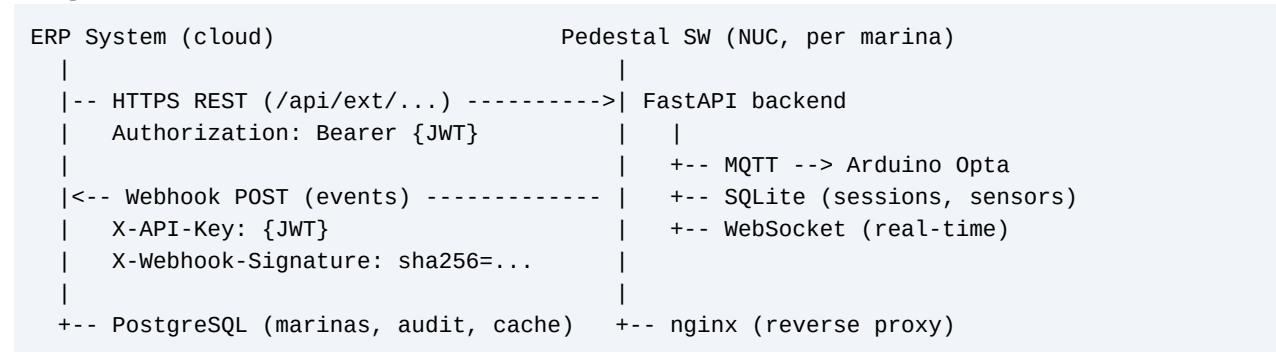
If `ERP_WEBHOOK_URL` is configured, the backend POSTs session updates to it (with the `X-API-Key` header) on three events: socket activated, a throttled telemetry update roughly every 60 s while active, and session ended — regardless of whether the stop came from the operator, the ERP, an unplug, or an automatic overload stop. The payload is the `GET /api/nfc/session` structure plus an event field. Delivery is fire-and-forget; failures are logged and never affect the session, MQTT flow, or operator control.

Note: Operator override is absolute. An NFC-started session is an ordinary session. The marina operator can stop it from the dashboard at any time; the ERP stop is best-effort and yields 409 if the operator already ended it.

1. Architecture Overview

The Cloud_IOT Pedestal SW is a full-stack IoT management platform deployed on an Intel NUC at each marina location. It manages physical pedestals (Arduino Opta controllers) that provide electricity (4 sockets) and water metering to boats. The system communicates locally via MQTT and exposes a REST API + WebSocket interface for real-time monitoring.

Integration Architecture:



The external API gateway (/api/ext/{path}) acts as a self-proxy: it validates the external JWT, generates a short-lived internal admin JWT, and forwards the request to the internal API. This ensures the external system never needs direct access to admin credentials.

Key Design Principles:

- **Single source of truth:** Pedestal SW owns all device data, sessions, and billing. The ERP system caches and aggregates but does not generate authoritative data.
- **Resilient integration:** ERP should implement retry + stale-cache fallback. Webhooks provide real-time push; polling provides on-demand pull.
- **Per-marina isolation:** Each NUC is independent. ERP manages multiple marinas by maintaining separate API clients with per-marina credentials.
- **Audit trail:** Both sides should log all control actions (allow, deny, stop) with user identity and timestamps for compliance.

2. Authentication

2.1 External API Key (Static JWT)

The primary authentication method for ERP integration. A long-lived JWT token is generated by the Pedestal SW admin and provided to the ERP system.

Property	Value
Algorithm	HS256
Lifetime	10 years (3,650 days)
JWT Claims	{ "sub": "external_api", "role": "external_api", "exp": ..., "iat": ... }
Generation	POST /api/admin/ext-api/config/rotate-key
Usage	Authorization: Bearer {api_key}

2.2 Service Account JWT (Short-Lived)

An alternative method using service account credentials. The ERP authenticates with email/password and receives a short-lived JWT. This is the method used by the ERP-IOT reference implementation.

```
POST /api/auth/service-token
Content-Type: application/json

{ "email": "service@marina.com", "password": "..." }

Response:
{ "access_token": "eyJ...", "token_type": "bearer" }
```

Property	Value
Lifetime	8 hours
JWT Claims	{ "sub": user_id, "email": "...", "role": "api_client", "exp": ... }
Refresh strategy	Cache for 7 hours, refresh 1 hour before expiry

2.3 Accepted Gateway Roles

The external API gateway accepts tokens with either role:

Role	Token Type	Use Case
external_api	Static 10-year JWT	Simple integration, single ERP
api_client	Short-lived service JWT	Multi-system integration, credential rotation

Note: Both roles provide the same endpoint access. The difference is lifecycle management. For pilot deployments, the static API key is simplest. For production with multiple integrating systems, service accounts provide better credential hygiene.

3. External API Gateway Setup

Before an ERP system can call Pedestal SW endpoints, an admin must configure and activate the API gateway. This is done through the admin UI (/api-gateway page) or directly via the admin API.

3.1 Configuration Workflow

Step 1 — Set allowed endpoints and events:

```
PUT /api/admin/ext-api/config
Authorization: Bearer {admin_jwt}
Content-Type: application/json

{
  "allowed_endpoints": [
    { "id": "pedestals.list",      "mode": "monitor" },
    { "id": "sessions.active",    "mode": "monitor" },
    { "id": "sessions.pending",  "mode": "monitor" },
    { "id": "controls.allow",     "mode": "bidirectional" },
    { "id": "controls.deny",      "mode": "bidirectional" },
    { "id": "controls.stop",      "mode": "bidirectional" },
    { "id": "analytics.daily",    "mode": "monitor" },
    { "id": "analytics.summary",  "mode": "monitor" },
    { "id": "alarms.active",      "mode": "monitor" }
  ],
  "webhook_url": "https://erp.example.com/api/webhooks/pedestal/1",
  "allowed_events": [
    "session_created", "session_updated", "session_completed",
    "power_reading", "water_reading", "temperature_reading"
  ]
}
```

Step 2 — Generate API key:

```
POST /api/admin/ext-api/config/rotate-key
Authorization: Bearer {admin_jwt}

Response: { "api_key": "eyJhbGciOiJIUzI1NiIs..." }
```

Step 3 — Verify configuration:

```
POST /api/admin/ext-api/config/verify
Authorization: Bearer {admin_jwt}

Response: { verified: true, results: { ... per-endpoint test results } }
```

Step 4 — Activate gateway:

```
POST /api/admin/ext-api/config/activate
Authorization: Bearer {admin_jwt}

(Requires verified=true from Step 3)
```

Note: Any change to `allowed_endpoints` or `allowed_events` resets `verified=false`, requiring re-verification before re-activation. This prevents misconfigured gateways.

3.2 Endpoint Access Modes

Mode	Allowed Methods	Description
monitor	GET only	Read-only access. ERP can query data but cannot trigger actions.
bidirectional	GET, POST, PUT, DELETE, PATCH	Full access. ERP can read data and trigger control actions.

4. Endpoint Catalog

The following 53 endpoints are exposed through the external API gateway, grouped by category exactly as they appear on the admin API Gateway page. Requests are sent to `https://<base>/api/ext/{path}` with the external JWT in the `Authorization: Bearer` header.

Note: For catalog paths that do not already begin with `/api/ext`, add that prefix when calling: e.g. `pedestals.list (/api/pedestals)` is called as `/api/ext/pedestals`. Paths already containing `/api/ext` are direct (non-proxied) routes. “Bi-dir = Yes” means the endpoint accepts write/control methods only when the admin enables it in `bidirectional` mode; in `monitor` mode every endpoint is read-only and a write returns 403.

4.1 Pedestals (1)

Endpoint ID	Method	Path	Bi-dir
pedestals.list	GET	/api/pedestals	No

4.2 Health (1)

Endpoint ID	Method	Path	Bi-dir
pedestals.health	GET	/api/pedestals/health	No

4.3 Sessions (2)

Endpoint ID	Method	Path	Bi-dir
sessions.active	GET	/api/sessions/active	No
sessions.pending	GET	/api/sessions/pending	No

4.4 Controls (18)

Endpoint ID	Method	Path	Bi-dir
controls.allow	POST	/api/controls/{id}/allow	Yes
controls.deny	POST	/api/controls/{id}/deny	Yes
controls.stop	POST	/api/controls/{id}/stop	Yes
controls.socket_approve	POST	/api/controls/sockets/{pedestal_id}/{socket_id}/approve	Yes
controls.socket_reject	POST	/api/controls/sockets/{pedestal_id}/{socket_id}/reject	Yes
controls.socket_cmd	POST	/api/controls/pedestal/{pedestal_id}/socket/{socket_name}/cmd	Yes
controls.water_cmd	POST	/api/controls/pedestal/{pedestal_id}/water/{valve_name}/cmd	Yes

Endpoint ID	Method	Path	Bi-dir
sockets.config_list	GET	/■api/■pedestals/■{pedestal_■id}/■sockets/■config	No
sockets.config_patch	PATCH	/■api/■pedestals/■{pedestal_■id}/■sockets/■{socket_■id}/■config	Yes
sockets.auto_log	GET	/■api/■pedestals/■{pedestal_■id}/■sockets/■{socket_■id}/■auto-activate-log	No
valves.config_list	GET	/■api/■pedestals/■{pedestal_■id}/■valves/■config	No
valves.config_patch	PATCH	/■api/■pedestals/■{pedestal_■id}/■valves/■{valve_■id}/■config	Yes
led_schedule.get	GET	/■api/■pedestals/■{pedestal_■id}/■led-schedule	No
led_schedule.upsert	PUT	/■api/■pedestals/■{pedestal_■id}/■led-schedule	Yes
led_schedule.delete	DELETE	/■api/■pedestals/■{pedestal_■id}/■led-schedule	Yes
led_schedule.test	POST	/■api/■pedestals/■{pedestal_■id}/■led-schedule/■test	Yes
controls.reset	POST	/■api/■controls/■pedestal/■{id}/■reset	Yes
controls.led	POST	/■api/■controls/■pedestal/■{id}/■led	Yes

4.5 Analytics (2)

Endpoint ID	Method	Path	Bi-dir
analytics.daily	GET	/■api/■analytics/■consumption/■daily	No
analytics.summary	GET	/■api/■analytics/■sessions/■summary	No

4.6 Alarms (2)

Endpoint ID	Method	Path	Bi-dir
alarms.active	GET	/■api/■alarms/■active	No
alarms.acknowledge	POST	/■api/■alarms/■{id}/■acknowledge	Yes

4.7 Berths (3)

Endpoint ID	Method	Path	Bi-dir
berths.list	GET	/■api/■berths	No
berths.occupancy_ext	GET	/■api/■ext/■pedestals/■{id}/■berths/■occupancy	No
berths.push_image_ext	POST	/■api/■ext/■pedestals/■{pedestal_■id}/■berths/■{berth_■id}/■reference-image	Yes

4.8 Diagnostics (1)

Endpoint ID	Method	Path	Bi-dir
diagnostics.run	POST	/api/pedestals/{id}/diagnostics/run	Yes

4.9 Camera (4)

Endpoint ID	Method	Path	Bi-dir
camera.snapshot	GET	/api/camera/{pedestal_id}/snapshot	No
camera.stream	GET	/api/camera/{pedestal_id}/stream	No
camera.frame_ext	GET	/api/ext/pedestals/{id}/camera/frame	No
camera.stream_ext	GET	/api/ext/pedestals/{id}/camera/stream	No

4.10 Load Monitoring (6)

Endpoint ID	Method	Path	Bi-dir
load.pedestal_get_ext	GET	/api/ext/pedestals/{pedestal_id}/load	No
load.socket_get_ext	GET	/api/ext/pedestals/{pedestal_id}/sockets/{socket_id}/load	No
load.marina_alarms_ext	GET	/api/ext/marinas/{marina_id}/load/alarms	No
load.pedestal_alarms_ext	GET	/api/ext/pedestals/{pedestal_id}/load/alarms	No
load.socket_history_ext	GET	/api/ext/pedestals/{pedestal_id}/sockets/{socket_id}/load/history	No
load.auto_stop_ack_ext	POST	/api/ext/pedestals/{pedestal_id}/sockets/{socket_id}/load/auto-stop/acknowledge	Yes

4.11 Usage History (3)

Endpoint ID	Method	Path	Bi-dir
usage.history	GET	/api/pedestals/{pedestal_id}/usage/history	No
usage.reports_list	GET	/api/pedestals/{pedestal_id}/usage/reports	No
usage.report_download	GET	/api/pedestals/{pedestal_id}/usage/reports/{month}	No

4.12 Mobile (3)

Endpoint ID	Method	Path	Bi-dir
mobile.qr_claim	POST	/api/mobile/qr/claim	Yes
mobile.session_live	GET	/api/mobile/sessions/{session_id}/live	No
mobile.socket_qr	GET	/api/mobile/socket/{pedestal_id}/{socket_id}/qr	No

4.13 QR (2)

Endpoint ID	Method	Path	Bi-dir
qr.pedestal_all	GET	/api/pedestals/{cabinet_id}/qr/all	No
qr.pedestal_regenerate	POST	/api/pedestals/{cabinet_id}/qr/regenerate	Yes

4.14 Breaker Management (5)

Endpoint ID	Method	Path	Bi-dir
breakers.pedestal_list_ext	GET	/api/ext/pedestals/{pedestal_id}/breakers	No
breakers.socket_get_ext	GET	/api/ext/pedestals/{pedestal_id}/sockets/{socket_id}/breaker	No
breakers.socket_reset_ext	POST	/api/ext/pedestals/{pedestal_id}/sockets/{socket_id}/breaker/reset	Yes
breakers.pedestal_history_ext	GET	/api/ext/pedestals/{pedestal_id}/breaker/history	No
breakers.marina_alarms_ext	GET	/api/ext/marinas/{marina_id}/breaker/alarms	No

5. Webhook Event Catalog

When a webhook URL is configured and events are activated, Pedestal SW POSTs these 38 event types to the ERP in real time. Events fire on every WebSocket broadcast (the same data the admin dashboard receives). The operator opts each event in on the API Gateway page; only enabled events are delivered.

5.1 All Events

Event ID	Name	Category	ERP use
power_reading	Power Readings	Sensors	Live consumption / monitoring
water_reading	Water Readings	Sensors	Live consumption / monitoring
temperature_reading	Temperature	Sensors	Live consumption / monitoring
moisture_reading	Moisture	Sensors	Live consumption / monitoring
heartbeat	Heartbeat	Health	Connectivity & maintenance
pedestal_health_updated	Pedestal Health	Health	Connectivity & maintenance
hardware_alarm	Hardware Alarm	Health	Connectivity & maintenance
session_created	Session Created	Sessions	Billing & session tracking
session_updated	Session Updated	Sessions	Billing & session tracking
session_completed	Session Completed	Sessions	Billing & session tracking
socket_pending	Socket Pending	Sessions	Billing & session tracking
socket_rejected	Socket Rejected	Sessions	Billing & session tracking
socket_state_changed	Socket State Changed	Sessions	Billing & session tracking
socket_auto_activate_skipped	Auto-Activate Skipped	Sessions	Billing & session tracking
session_telemetry	Session Telemetry (mobile)	Sessions	Billing & session tracking
session_ended	Session Ended (mobile)	Sessions	Billing & session tracking
pedestal_registered	Pedestal Registered	Discovery	Provisioning
user_plugged_in	User Plugged In	Sessions	Billing & session tracking

Event ID	Name	Category	ERP use
invoice_created	Invoice Created	Billing	Invoicing
berth_occupancy_updated	Berth Occupancy	Berths	Marina & berth management
diagnostics_result	Diagnostics Results	Diagnostics	Maintenance
marina_door	Cabinet Door	Hardware	Diagnostics & audit
marina_event	Cabinet Events	Hardware	Diagnostics & audit
marina_ack	Command Acks	Hardware	Diagnostics & audit
opta_socket_status	Opta Socket Status	Hardware	Diagnostics & audit
opta_water_status	Opta Water Status	Hardware	Diagnostics & audit
opta_status	Opta Status	Hardware	Diagnostics & audit
pedestal_reset_sent	Pedestal Reset	Controls	Audit trail
breaker_state_changed	Breaker State Changed	Breaker Management	Circuit safety
breaker_alarm	Breaker Alarm	Breaker Management	Circuit safety
valve_flow_warning	Valve Flow Warning	Hardware	Diagnostics & audit
led_changed	LED State Changed	Hardware	Diagnostics & audit
hardware_config_updated	Hardware Config Updated	Load Monitoring	Overload protection
meter_load_warning	Meter Load Warning	Load Monitoring	Overload protection
meter_load_critical	Meter Load Critical	Load Monitoring	Overload protection
meter_load_resolved	Meter Load Resolved	Load Monitoring	Overload protection
meter_load_auto_stop	Meter Load Auto-Stop	Load Monitoring	Overload protection
meter_load_auto_stop_acknowledged	Auto-Stop Acknowledged	Load Monitoring	Overload protection

5.2 Webhook Delivery

Property	Value
Method	POST
Content-Type	application/json
Timeout	5 seconds (fire-and-forget)
Retry	None (single attempt per event)
Authentication	X-API-Key: {api_key_jwt}
Cache	Config cached 30 seconds (invalidated on config change)

Note: Webhook delivery is fire-and-forget with a 5-second timeout. The ERP system must respond quickly (HTTP 2xx) or the event is dropped. For reliability, implement a combination of webhooks (push) and API polling (pull) as the reference implementation does.

6. Core Integration Flows

6.1 Dashboard / Monitoring (Pull)

The ERP system periodically polls Pedestal SW endpoints to build a dashboard view. This is supplemented by webhook events for real-time updates.

```
# 1. List pedestals and their state
GET /api/ext/pedestals
Authorization: Bearer {api_key}

# 2. Get health overview
GET /api/ext/pedestals/health

# 3. Get active sessions (with consumption data)
GET /api/ext/sessions/active

# 4. Get pending sessions (awaiting approval)
GET /api/ext/sessions/pending

# 5. Get active alarms
GET /api/ext/alarms/active
```

The ERP-IOT reference implementation fetches all 4 data points in parallel (async) and merges them into a single dashboard response. On failure, it falls back to cached data and marks the response as stale.

6.2 Session Control (Push)

When an operator in the ERP system needs to approve, deny, or stop a session:

```
# Approve a pending session
POST /api/ext/controls/{session_id}/allow
Authorization: Bearer {api_key}

# Deny a session (with optional reason)
POST /api/ext/controls/{session_id}/deny
Body: { "reason": "Berth not assigned" }

# Stop an active session
POST /api/ext/controls/{session_id}/stop
Authorization: Bearer {api_key}
```

Note: These endpoints require mode: `bidirectional` in the gateway configuration.

6.3 Real-Time Events (Webhook Push)

Configure the ERP webhook endpoint to receive real-time events. The ERP should:

- Validate the X-API-Key header matches the expected API key
- Optionally validate HMAC signature via X-Webhook-Signature (if `webhook_secret` is configured on both sides)
- Persist events to a local database (`session_log`, `alarm_log`, etc.)
- Update cached dashboard data based on event type
- Broadcast to connected UI clients via WebSocket

- Return HTTP 200 within 5 seconds

6.4 Energy Analytics

```
# Daily consumption (with date range)
GET /api/ext/analytics/consumption/daily?start=2026-04-01&end;=2026-04-16

# Session summary (totals and averages)
GET /api/ext/analytics/sessions/summary
```

6.5 Berth Occupancy & Camera (Marina View)

These are the marina-facing pull endpoints used in the MVP Pilot (see the MVP Pilot section). Berth occupancy is also delivered in real time via the `berth_occupancy_updated` webhook event.

```
# Occupancy for all berths under a pedestal (numeric id or Opta client id)
GET /api/ext/pedestals/{pedestal_id}/berths/occupancy

# Live camera stream URL for the pedestal that covers the berth
GET /api/ext/pedestals/{pedestal_id}/camera/stream

# Single live JPEG frame, captured by the NUC (no RTSP needed by the ERP)
GET /api/ext/pedestals/{pedestal_id}/camera/frame
```

Note: Each capability has its own gateway toggle (`berths.occupancy_ext`, `camera.stream_ext`, `camera.frame_ext`) and returns 503 `{error, reason}` when disabled or the feature is unavailable.

7. Session Lifecycle & Billing

7.1 Session State Machine

```
Customer requests power/water
|
v
[ PENDING ] ---deny---> [ DENIED ]
|
allow
|
v
[ ACTIVE ] ---stop/complete---> [ COMPLETED ]
|                               |
+-- energy_kwh accumulating    +-- final readings frozen
+-- water_liters accumulating   +-- invoice generated
```

7.2 Billing Data Flow

Billing is calculated by the Pedestal SW based on configured rates. The ERP should consume this data, not recalculate it.

Data Point	Source	ERP Action
Billing rates	GET /api/billing/config (admin)	Display, optionally mirror to ERP billing module
Session consumption	session_completed webhook event	Record energy_kwh, water_liters, calculate costs
Invoice	Generated on session completion	Sync via /api/customer/invoices or webhook
Spending summary	GET /api/billing/spending	Aggregate for reporting
Per-session detail	GET /api/billing/spending/detail	Detailed breakdown for accounting

7.3 Billing Configuration

```
# Current rates
GET /api/ext/billing/config

Response:
{
  "kwh_price_eur": 0.30,
  "liter_price_eur": 0.015
}
```

Note: Default rates are seeded on first startup (0.30 EUR/kWh, 0.015 EUR/liter). Rates can be changed by admin via the UI or API. The ERP should periodically sync these rates rather than hardcoding them.

7.4 Invoice Model

Field	Type	Description
id	int	Invoice primary key

Field	Type	Description
session_id	int	Associated session
customer_id	int	Customer who consumed
energy_kwh	float	Total electricity consumed
water_liters	float	Total water consumed
energy_cost_eur	float	Electricity cost (energy_kwh x rate)
water_cost_eur	float	Water cost (water_liters x rate)
total_eur	float	energy_cost + water_cost
paid	0/1	Payment status (0=unpaid, 1=paid)
created_at	datetime	Invoice creation timestamp

8. Contracts & Invoices

8.1 Contract Templates

Admins create contract templates that customers must sign before using marina services. Templates define terms, conditions, and validity period.

Endpoint	Method	Description
/api/contracts/templates	GET	List all active templates
/api/contracts/templates	POST	Create new template (admin)
/api/contracts/templates/{id}	PATCH	Update template (admin)
/api/admin/contracts	GET	List all signed contracts (admin)
/api/admin/contracts/{id}/pdf	GET	Download contract PDF (admin)

8.2 Customer Contract Flow

Endpoint	Method	Description
/api/customer/contracts/pending	GET	Templates not yet signed by customer
/api/customer/contracts/{template_id}/sign	POST	Sign with base64 signature image
/api/customer/contracts/mine	GET	Customer's signed contracts
/api/customer/contracts/{id}/pdf	GET	Download signed contract PDF

8.3 Invoice Endpoints

Endpoint	Method	Description
/api/customer/invoices/mine	GET	Customer's invoice list
/api/customer/invoices/{id}/pay	POST	Mark invoice as paid
/api/customer/invoices/{id}/pdf	GET	Download invoice PDF
/api/billing/spending	GET	Aggregate spending by customer (admin)
/api/billing/spending/detail	GET	Per-session cost breakdown (admin)

Note: For ERP integration, the admin billing endpoints (/api/billing/*) are most relevant. These provide aggregated spending data suitable for importing into ERP accounting modules. Invoice PDFs can be downloaded and archived in the ERP document store.

9. Webhook Payload Reference

All webhook payloads follow the same envelope structure:

```
{
  "event": "event_id",
  "data": { ... event-specific payload ... },
  "timestamp": "2026-04-16T12:34:57Z"
}
```

9.1 session_created

```
{
  "event": "session_created",
  "data": {
    "session_id": 123,
    "pedestal_id": 1,
    "socket_id": 2,
    "type": "electricity",
    "status": "active",
    "started_at": "2026-04-16T12:34:56Z",
    "customer_id": 5,
    "customer_name": "John Doe"
  },
  "timestamp": "2026-04-16T12:34:57Z"
}
```

9.2 session_completed

```
{
  "event": "session_completed",
  "data": {
    "session_id": 123,
    "pedestal_id": 1,
    "socket_id": 2,
    "type": "electricity",
    "status": "completed",
    "energy_kwh": 2.45,
    "water_liters": 0.0,
    "customer_id": 5
  },
  "timestamp": "2026-04-16T14:22:10Z"
}
```

9.3 power_reading

```
{
  "event": "power_reading",
  "data": {
    "pedestal_id": 1,
    "socket_id": 2,
    "voltage": 230.5,
    "current": 4.2,
    "power_w": 968.1,
    "energy_kwh": 1.23,
    "timestamp": "2026-04-16T13:00:00Z"
  }
}
```

```

    },
    "timestamp": "2026-04-16T13:00:01Z"
  }

```

9.4 temperature_reading / moisture_reading

```

{
  "event": "temperature_reading",
  "data": {
    "pedestal_id": 1,
    "value": 47.2,
    "severity": "warning",
    "alarm": true,
    "temp_sensor_reachable": true,
    "last_temp_sensor_check": "2026-06-14T13:05:00Z",
    "timestamp": "2026-06-14T13:05:00Z"
  },
  "timestamp": "2026-06-14T13:05:01Z"
}

```

Temperature is read from the Papouch TME sensor by the NUC every 30 s and evaluated against two-sided range alarms with clearing hysteresis (1°C):

Band	Threshold	severity
High critical	$\geq 60\text{ °C}$	critical
High warning	$\geq 45\text{ °C}$	warning
Normal	$0\text{ °C} - 45\text{ °C}$	null
Low warning	$\leq 0\text{ °C}$	warning
Low critical	$\leq -10\text{ °C}$	critical

Note: severity is null in the normal band; alarm is true whenever severity is non-null. temperature_reading is push-only (no pull endpoint). Moisture alarm triggers at >90%. When severity: "critical", the ERP should log this as a critical event.

10. Error Handling & Resilience

10.1 Gateway Error Codes

HTTP Code	Meaning	ERP Action
200	Success	Process response normally
401	Invalid/expired JWT	Refresh token (service account) or alert admin (API key)
403	Endpoint not allowed or monitor-mode POST	Check gateway config; do not retry
404	Unknown endpoint path	Check endpoint catalog; do not retry
502	Pedestal internal error (upstream failure)	Retry with backoff; fall back to cache
503	Gateway not activated	Alert admin to activate gateway

10.2 Recommended Retry Strategy

Based on the ERP-IOT reference implementation:

Parameter	Value	Rationale
Max retries	3	Balance between reliability and latency
Backoff	Exponential: 1s, 2s, 4s	Avoid overwhelming a recovering service
Retry on	Connection errors, timeouts, 5xx	Transient failures
No retry on	4xx (except 401)	Client errors won't resolve on retry
Request timeout	10 seconds	NUC is local network; fast responses expected
Token refresh on 401	Auto-refresh and retry once	Token may have expired

10.3 Stale Data Pattern

When the Pedestal SW is unreachable (NUC offline, network issue), the ERP should:

- Return the last successfully cached response
- Mark the response with an `is_stale: true` flag
- Display a visual indicator in the UI (e.g., yellow banner)
- Continue retrying in the background
- Log the failure to a `sync_log` table for diagnostics

11. Security Considerations

11.1 Transport Security

- **HTTPS required** for production. The NUC uses nginx with optional TLS or Cloudflare tunnel.
- **Webhook URL** must be HTTPS in production to protect event payloads in transit.
- **API keys** should never be logged, displayed in UIs, or stored in plaintext config files.

11.2 Credential Management

- **API key rotation:** Use POST /api/admin/ext-api/config/rotate-key to generate a new key. The old key is immediately invalidated.
- **Service account passwords:** Encrypt at rest using Fernet (AES-128-CBC + HMAC-SHA256) as the reference implementation does.
- **JWT secret:** Must match between Pedestal SW and any system validating its tokens. Minimum 32 bytes of randomness.

11.3 Webhook Validation

The ERP should validate incoming webhooks to prevent spoofed events:

```
import hmac, hashlib

def verify_webhook(payload_bytes, signature_header, secret):
    expected = "sha256=" + hmac.new(
        secret.encode(), payload_bytes, hashlib.sha256
    ).hexdigest()
    return hmac.compare_digest(expected, signature_header)
```

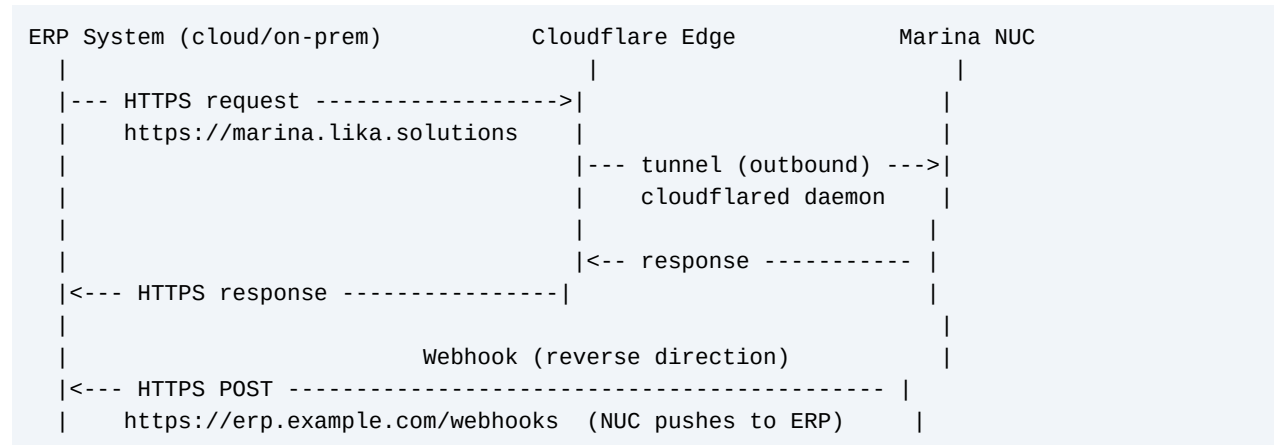
11.4 Access Control

- **Principle of least privilege:** Only enable endpoints the ERP actually needs.
- **Use monitor mode** for read-only endpoints; only enable bidirectional for control actions.
- **Separate API keys** per integrating system if multiple systems connect.
- **Audit all control actions** with user identity and timestamps.

12. Network & Cloudflare Configuration

The Pedestal SW runs on an Intel NUC at the marina with no static public IP address. Internet access is provided via a 5G/LTE router with dynamic IP. All external access to the NUC is routed through a **Cloudflare Tunnel**, managed by Lika Digital. This section explains the network architecture and what the ERP system provider must supply to Lika Digital to enable connectivity.

12.1 Current Network Architecture



The NUC maintains a persistent outbound connection to Cloudflare (via `cloudflared` daemon). No inbound ports are opened on the marina's firewall. Cloudflare terminates TLS and routes traffic through the tunnel to nginx on the NUC (port 80), which proxies to the FastAPI backend.

12.2 Pedestal SW Access (ERP → NUC)

The ERP system accesses the Pedestal SW API through the Cloudflare tunnel hostname. This is already configured and operational:

Property	Value
Base URL	https://marina.likasolutions
API Gateway	https://marina.likasolutions/api/ext/{path}
TLS	Terminated by Cloudflare (automatic, managed certificate)
Authentication	Bearer JWT in Authorization header (see Section 2)
Tunnel ID	0c795485-c696-49a9-973c-55b2aab98c39

Note: The ERP system does not need to know the NUC's IP address. All API requests go to <https://marina.likasolutions/api/ext/...> and Cloudflare handles routing through the tunnel. TLS is automatic.

12.3 MarinaMaster ERP — Cloudflare Tunnel Setup

For the Pedestal SW to send webhook events to MarinaMaster, and for MarinaMaster to be reachable at a stable HTTPS URL, there are two options depending on whether MarinaMaster has a static public IP address.

Option A — MarinaMaster Has a Static Public IP

If MarinaMaster has a static public IP address, **Lika Digital handles the entire setup**. MarinaMaster only needs to provide:

- Their server's public IP address
- The port their ERP application runs on (e.g., 8000, 3000)

Lika Digital will configure the Cloudflare tunnel and DNS record on their side. **No action required from MarinaMaster.**

Option B — MarinaMaster Does NOT Have a Static Public IP

If MarinaMaster does not have a static public IP, they need to install a small tunneling agent called `cloudflared` on their server. This is a free tool provided by Cloudflare and takes approximately 5 minutes to set up.

Step 1 — Download and install cloudflared.

Download the installer for your platform from:

<https://github.com/cloudflare/cloudflared/releases/latest>

Available for Windows, Linux, and macOS.

Step 2 — Log in to Cloudflare.

Open a terminal on the server and run:

```
cloudflared tunnel login
```

A browser window will open asking you to log in to a Cloudflare account. If MarinaMaster does not have a Cloudflare account, create a free one at cloudflare.com.

Step 3 — Create a tunnel.

Run the following command:

```
cloudflared tunnel create marinamaster-tunnel
```

This will generate a tunnel ID that looks like `xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxxx`. **Copy this ID — you will send it to Lika Digital.**

Step 4 — Start the tunnel.

Replace `PORT` with the port your ERP application runs on (e.g., 8000 or 3000):

```
cloudflared tunnel run --url http://localhost:PORT marinamaster-tunnel
```

Step 5 — Send the tunnel ID to Lika Digital.

Lika Digital will add a DNS record pointing `marinamaster.lika.solutions` to your tunnel. Once done, the ERP application will be accessible at `https://marinamaster.lika.solutions`.

Step 6 — Make the tunnel start automatically on server restart.

Run the following commands:

```
cloudflared service install
systemctl enable cloudflared
systemctl start cloudflared
```

Note: Note for Lika Digital (internal): Once MarinaMaster provides the tunnel ID, add a CNAME DNS record in the Cloudflare dashboard for `lika.solutions` pointing `marinamaster` to `{tunnel_id}.cfargotunnel.com` with proxy enabled (orange cloud). No changes to the NUC

configuration are needed for Option B.

Summary — Webhook URL After Setup

Option	Webhook URL configured in Pedestal SW	Who sets it up
A (static IP)	<code>https://marinamaster.lika.solutions/api/webhooks/...</code>	Lika Digital
B (cloudflared)	<code>https://marinamaster.lika.solutions/api/webhooks/...</code>	MarinaMaster installs cloudflared, Lika adds DNS

In both cases, the final webhook URL uses the same `marinamaster.lika.solutions` domain, with HTTPS provided automatically by Cloudflare.

12.4 Information ERP Provider Must Supply to Lika Digital

To enable integration, the ERP system management team must provide the following information to Lika Digital (the Cloudflare account owner):

Item	Required?	Description	Example
Webhook URL	YES	Public HTTPS endpoint where the NUC should send real-time events. Must respond within 5 seconds.	<code>https://erp.example.com/api/webhooks/pedestal/1</code>
Webhook auth token	YES	Shared secret or token that Lika will configure in the gateway. The NUC sends this as X-API-Key header on every webhook POST.	Agreed during setup
Service account email	YES	Email address to use for the ERP's service account on Pedestal SW. Lika Digital will create this account with <code>role=api_client</code> .	<code>erp-service@partner.com</code>
Service account password	YES	Password for the service account. ERP must store this securely (encrypted at rest). Used to obtain short-lived JWT tokens.	Agreed during setup
Required endpoints	YES	List of endpoint IDs from the catalog (Section 4) that the ERP needs access to, with mode (monitor or bidirectional).	<code>sessions.active (monitor)</code> , <code>controls.allow (bidirectional)</code>
Required events	YES	List of webhook event IDs from the catalog (Section 5) the ERP wants to receive.	<code>session_created</code> , <code>session_completed</code> , <code>power_reading</code>
IP whitelist	OPTIONAL	If the ERP has a static IP range, Lika can add a Cloudflare Access policy to restrict API access to those IPs only.	<code>203.0.113.0/24</code>
mTLS certificate	OPTIONAL	For maximum security, the ERP can provide a client TLS certificate. Lika configures Cloudflare Access to require it on the tunnel.	X.509 PEM certificate

12.5 What Lika Digital Configures

Once the ERP provider supplies the above information, Lika Digital will:

- **Create a service account** on the Pedestal SW with role `api_client` using the agreed email and password.
- **Configure the API Gateway** with the requested endpoints, modes, webhook URL, and events.
- **Verify and activate** the gateway configuration.
- **Optionally add Cloudflare Access policy** to restrict tunnel access to the ERP's IP range or mTLS certificate.
- **Provide the API base URL** to the ERP team: `https://marina.lika.solutions`.
- **Test end-to-end** connectivity with the ERP team before go-live.

12.6 DNS & TLS (Managed by Lika Digital)

All DNS and TLS configuration is handled by Lika Digital. The ERP team does **not** need to configure any DNS records or TLS certificates. For reference:

Component	Configuration	Managed By
Domain	<code>*.lika.solutions</code>	Lika Digital (Cloudflare DNS)
TLS Certificate	Automatic via Cloudflare (edge certificate)	Lika Digital
Tunnel daemon	<code>cloudflared</code> on NUC (systemd service)	Lika Digital
Ingress rules	Per-hostname routing in tunnel config	Lika Digital
Access policies	Optional IP whitelist or mTLS enforcement	Lika Digital

12.7 Multiple Marina Sites

Each marina has its own NUC with its own Cloudflare tunnel and subdomain. If the ERP integrates with multiple marinas, it will use separate base URLs and separate service account credentials for each:

```
# Marina 1
Base URL:      https://marina-1.lika.solutions
Service account: erp-service@partner.com / {password_1}

# Marina 2
Base URL:      https://marina-2.lika.solutions
Service account: erp-service@partner.com / {password_2}

# Each marina is independent – separate NUC, separate DB, separate credentials.
```

13. Multi-Client Authentication (Service Accounts)

When multiple external systems need to integrate with the same Pedestal SW instance (e.g., an ERP system and a separate analytics platform), each system should authenticate with its own **service account**. This provides credential isolation, independent revocation, and per-client audit trails.

13.1 Why Not Share the Static API Key?

The external API gateway supports a single static API key (Section 2.1). While simple, sharing this key between multiple systems has significant drawbacks:

Problem	Impact
No audit trail separation	Cannot distinguish which system made a request in logs
All-or-nothing revocation	Rotating the key invalidates ALL integrations simultaneously
Single point of compromise	If one system leaks the key, all systems are exposed
No per-client permissions	All systems share the same endpoint access configuration

13.2 Service Account Architecture

Instead, each ERP or external system gets its own service account with role `api_client`. The authentication flow:

```
# Step 1: ERP authenticates with service credentials (once per 8 hours)
POST https://marina.lika.solutions/api/auth/service-token
Content-Type: application/json

{
  "email": "erp-service@partner.com",
  "password": "securePassword123"
}

Response:
{
  "access_token": "eyJhbGciOiJIUzI1NiIs... ",
  "role": "api_client",
  "email": "erp-service@partner.com"
}

# Step 2: ERP uses the JWT for all API calls (valid 8 hours)
GET https://marina.lika.solutions/api/ext/sessions/active
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...

# Step 3: ERP refreshes token before expiry (recommended: every 7 hours)
# Simply repeat Step 1 to get a new token.
```

13.3 Token Lifecycle Management

The ERP should implement token caching and refresh logic:

Parameter	Value	Notes
Token lifetime	8 hours	Set by Pedestal SW (JWT exp claim)

Parameter	Value	Notes
Recommended cache	7 hours	Refresh 1 hour before expiry to avoid request failures
On 401 response	Re-authenticate immediately	Token may have been revoked or clock skew
Storage	In-memory (preferred) or encrypted at rest	Never store JWT in plaintext files or logs
Concurrency	Cache per-client, not globally	Each marina/integration gets its own token

```
# Pseudocode: Token caching pattern
class PedestalClient:
    def __init__(self, base_url, email, password):
        self.base_url = base_url
        self.email = email
        self.password = password
        self._token = None
        self._token_expires = None

    def _get_token(self):
        if self._token and datetime.utcnow() < self._token_expires:
            return self._token
        resp = httpx.post(f'{self.base_url}/api/auth/service-token',
                        json={'email': self.email, 'password': self.password})
        resp.raise_for_status()
        self._token = resp.json()['access_token']
        self._token_expires = datetime.utcnow() + timedelta(hours=7)
        return self._token

    def get_sessions(self):
        token = self._get_token()
        return httpx.get(f'{self.base_url}/api/ext/sessions/active',
                        headers={'Authorization': f'Bearer {token}'})
```

13.4 Service Account Setup (Performed by Lika Digital)

Service accounts are created by a Pedestal SW admin. The process:

- **Step 1:** ERP provider sends Lika Digital the desired service account email and password.
- **Step 2:** Lika Digital creates the account on the NUC via the admin API or directly in the database with role `api_client`.
- **Step 3:** Lika Digital configures the API Gateway endpoints and events for this integration.
- **Step 4:** ERP team tests authentication: POST `/api/auth/service-token` with their credentials.
- **Step 5:** ERP team begins calling gateway endpoints with the obtained JWT.

Note: Service accounts use role `api_client` and can only authenticate via `/api/auth/service-token`. They cannot use the admin UI login (which requires authenticator-app TOTP two-factor). The service-token endpoint returns a JWT directly, with no 2FA step, making it suitable for automated system-to-system integration.

13.5 Multiple ERP Systems on One Pedestal

If multiple ERP systems connect to the same marina's Pedestal SW:

Feature	Current Support	Recommendation
Separate service accounts	YES — each ERP gets its own api_client user	Create one per integrating system
Separate endpoint permissions	PARTIAL — gateway config is global (single row)	All service accounts share the same allowed endpoints
Separate webhook URLs	NO — single webhook_url in config	ERP systems should use polling + one shared webhook URL that fans out internally
Audit by client	YES — JWT contains email/sub, logged in gateway	Each service account's actions are distinguishable in logs

Note: For the pilot phase, a single ERP integration with one service account is the expected setup. Multi-client support with per-client endpoint permissions and multiple webhook URLs can be added in a future release if demand requires it.

14. Reference Implementation (ERP-IOT)

The ERP-IOT project is a working reference implementation that demonstrates all integration patterns described in this document. It is deployed and tested against the Pedestal SW.

14.1 Technology Stack

Component	Technology	Purpose
Backend	FastAPI (Python 3.11+)	REST API, webhook handling, Pedestal API client
Frontend	React + TypeScript	Multi-marina dashboard, controls, analytics
Database	PostgreSQL 16	Marinas, users, cache, audit logs
Auth	JWT (HS256) + bcrypt	User auth + service account auth to Pedestal
Encryption	Fernet (AES-128-CBC)	Service account password storage
Deployment	Docker Compose + nginx	Orchestration and reverse proxy

14.2 Key Design Patterns

Per-Marina API Client Factory:

A singleton factory creates one `PedestalAPIClient` per marina. Each client manages its own JWT token lifecycle (authenticate on first call, cache token, refresh before expiry). This ensures efficient connection reuse.

Stale Data Fallback:

Every API call to Pedestal SW is wrapped in retry logic. On total failure, the system falls back to the last cached response from the `pedestal_cache` table. Responses include an `is_stale` boolean so the frontend can display a degraded-mode indicator.

Webhook Reception + WebSocket Broadcast:

Incoming webhooks from Pedestal SW are validated (HMAC), persisted to typed log tables (`alarm_log`, `session_log`), and immediately broadcast to connected WebSocket clients scoped by `marina_id`.

Comprehensive Audit Trail:

Every control action (allow, deny, stop, acknowledge alarm) is recorded in an `audit_log` table with: `user_id`, `marina_id`, `pedestal_id`, action name, `target_id`, JSON details, and timestamp.

14.3 Data Model Summary

Table	Purpose	Key Fields
marinas	Marina configuration	<code>api_base_url</code> , <code>service_email</code> , <code>encrypted_password</code> , <code>webhook_secret</code>
users	ERP operator accounts	<code>email</code> , <code>password_hash</code> , <code>role</code> (<code>super_admin</code> / <code>marina_manager</code>)
<code>user_marina_access</code>	RBAC junction table	<code>user_id</code> , <code>marina_id</code> , <code>granted_by</code>
<code>pedestal_cache</code>	Cached Pedestal data	<code>marina_id</code> , <code>pedestal_id</code> , <code>last_seen_data</code> (JSON), <code>is_stale</code>

Table	Purpose	Key Fields
alarm_log	Alarm event history	marina_id, pedestal_id, alarm_data (JSON), acknowledged_at
session_log	Session event history	marina_id, pedestal_id, session_data (JSON)
sync_log	API call audit	marina_id, sync_type, status, error_message
audit_log	Control action audit	user_id, marina_id, action, target_id, details (JSON)

15. Quick-Start Checklist

Follow these steps to integrate a new ERP system with the Pedestal SW:

- 1. Network access** — Ensure the ERP can reach the NUC's IP address (or Cloudflare tunnel URL) on port 80/443.
- 2. Configure gateway** — Admin logs into Pedestal UI → API Gateway page → selects endpoints + events → sets webhook URL.
- 3. Generate API key** — Click 'Rotate Key' in the gateway page. Copy the JWT and store it securely in the ERP configuration.
- 4. Verify & activate** — Click 'Verify' to test connectivity, then 'Activate'.
- 5. Implement authentication** — ERP sends `Authorization: Bearer {api_key}` on every request.
- 6. Implement polling** — Periodically call `GET /api/ext/pedestals`, `GET /api/ext/sessions/active`, etc.
- 7. Implement webhook handler** — Accept POST events at the configured URL. Validate the `X-API-Key` header. Return 200 quickly.
- 8. Implement stale-data fallback** — Cache successful responses. On failure, serve cached data with a stale indicator.
- 9. Implement control actions** — If needed, call `POST /api/ext/controls/{id}/allow|deny|stop` for session management.
- 10. Implement billing sync** — Listen for `session_completed` events. Query `/api/ext/billing/spending/detail` for reconciliation.
- 11. Audit logging** — Log all control actions with operator identity and timestamps.
- 12. Test end-to-end** — Use the pedestal simulator to generate sessions and verify the full flow from event → webhook → ERP processing.

16. Data for the Marina Operator Persona

Inside MarinaMaster, the **Marina Operator** is the staff role that runs the site day to day. This section maps each operator need to the Pedestal SW data that satisfies it, so MarinaMaster knows which endpoints to poll and which webhook events to subscribe to when building the operator console. The operator console is the read/write surface — it both displays state and issues control actions.

16.1 Core Data Objects Exchanged

The following objects are the primary structures exchanged over the API and webhooks. Field-level payloads are in Section 9 (events), Section 7.4 (invoice) and the endpoint responses.

Object	Key fields	Primary source
Pedestal	id, name, location, online, initialized, mobile_enabled, sockets[]	pedestals.list / pedestals.health

Object	Key fields	Primary source
Session	id, pedestal_id, socket_id, type (electricity/water), status, started_at, ended_at, energy_kwh, water_liters, customer_id	sessions.active / sessions.pending / session_* events
Socket load	socket_id, phases, rated_amps, current_amps, load_pct, status (normal/warning/critical), meter_type	load.socket_get_ext / meter_load_* events
Breaker	socket_id, state, trip_cause, type, rating, poles, rcd, trip_count	breakers.* / breaker_state_changed, breaker_alarm
Alarm	id, kind, pedestal_id, socket_id, value, threshold, acknowledged, triggered_at	alarms.active / temperature_reading, moisture_reading, hardware_alarm
Invoice	id, session_id, customer_id, energy_kwh, water_liters, total_eur, paid	invoice_created event / billing endpoints

16.2 Operator Need → Pedestal Data Mapping

Operator need	Endpoints (pull)	Events (push)
Live site overview & connectivity	pedestals.list, pedestals.health	heartbeat, pedestal_health_updated
Who is drawing power / water now	sessions.active	session_created, session_updated, session_telemetry
Pending requests to approve	sessions.pending	socket_pending, user_plugged_in
Allow / deny / stop a session	controls.allow, controls.deny, controls.stop (bidirectional)	session_updated, session_completed, socket_rejected
Load & overload protection	load.pedestal_get_ext, load.socket_get_ext, load.*_alarms_ext, load.auto_stop_ack_ext	meter_load_warning, meter_load_critical, meter_load_auto_stop, meter_load_resolved
Breaker status & reset	breakers.pedestal_list_ext, breakers.socket_get_ext, breakers.socket_reset_ext	breaker_state_changed, breaker_alarm
Environmental & security alarms	alarms.active, alarms.acknowledge	temperature_reading, moisture_reading, hardware_alarm, marina_door
Run diagnostics / commission	diagnostics.run, pedestals.list	diagnostics_result, hardware_config_updated, pedestal_registered
Camera, occupancy & berths	camera.frame_ext, camera.stream_ext, berths.list, berths.occupancy_ext	berth_occupancy_updated
LED, schedule & reset	controls.led, led_schedule.*, controls.reset (bidirectional)	led_changed, pedestal_reset_sent
Energy analytics & billing reconciliation	analytics.daily, analytics.summary	session_completed, invoice_created

Note: All control actions (allow, deny, stop, acknowledge, reset, breaker reset) require **bidirectional** mode and should be recorded in MarinaMaster's audit log with operator identity and timestamp. Read-only

monitor staff in MarinaMaster should be given only the pull endpoints and events, never the control endpoints.

17. Data Exposed to the MyMarina End-User App

MyMarina is MarinaMaster's mobile application for boat owners (end users). MarinaMaster acts as the broker: it consumes the full operator-grade data from Pedestal SW (Section 16) and re-exposes a **curated, customer-scoped subset** to MyMarina. MyMarina never talks to operator control endpoints and only ever sees the signed-in customer's own data.

17.1 MyMarina Feature → Underlying Data

MyMarina feature	Underlying pedestal data	Source
Start a session by scanning the socket QR	Claim the socket for this customer; bind customer_id to the session	mobile.qr_claim, mobile.socket_qr
Live meter while charging / taking water	duration, energy_kwh, power_kw (electricity) or litres (water) — own session only	mobile.session_live, session_telemetry event
My session history	completed sessions filtered to this customer_id, with consumption	sessions + session_completed (customer-scoped by MarinaMaster)
My invoices & payment status	energy/water cost, total_eur, paid flag, invoice PDF	invoice_created event + billing data
My contracts	pending agreements to sign, signed agreements + PDF	customer contract data (MarinaMaster-side)
Service requests	request a marina service with notes; status updates	service-order data (MarinaMaster-side)
Support chat & notifications	messages to/from staff; push on approval, reply, alarm-affecting-me	chat data + push tokens (MarinaMaster-side)

17.2 Scoping & Privacy Rules

- **Customer scoping is mandatory.** MyMarina must filter every list (sessions, invoices, contracts) to the authenticated customer_id. Pedestal SW session and invoice records carry customer_id for exactly this purpose.
- **No control surface.** Do not expose allow / deny / stop / reset / breaker / LED endpoints to MyMarina. End users start and stop only their *own* session via the mobile/QR endpoints.
- **Auto-activate sessions are invisible to MyMarina.** A plug-and-go (auto-activate) session is created with customer_id = null, so it has no owner and must not appear in any customer's history or invoices. Only QR/app-claimed sessions are billable to a MyMarina account.
- **Data minimization.** MyMarina should receive only the fields a boat owner needs (their consumption, cost, status) — not raw operator telemetry, other berths, camera frames, or site-wide alarms.

Note: The split is clean: Section 16 (operator) is the full read/write operational surface; Section 17 (MyMarina) is a narrow, per-customer, read-mostly slice. MarinaMaster owns the boundary — Pedestal SW provides the data and the customer_id; MarinaMaster enforces which customer sees what.